

Plant Health

An AI-Powered Mobile Application for Tomato Leaf Disease Diagnosis

Submitted by

Name	ID
Habiba	0272130005101046
Mithila Akter	0272130005101048
Imran Hossain Badsha	0272130005101012
Alfe Sani	1935202085

Supervised By Faria Rahman Annasha, Lecturer, Department of CSE City University of Bangladesh

Table of Contents

1. Introduction
2. Background Study
3. Problem Statement
4. Objectives
5. Literature Review
6. System Overview
7. Methodology
8. Technology Stack
9. System Design & Architecture
10. Implementation Details
11. Results and Evaluation
12. Working Schedule
13. Limitations
14. Expected Outcome and Future Work

15. Conclusion

16. References

1. Introduction

Tomato is one of the most widely grown vegetable crops, but its yield is often damaged by leaf diseases such as Early Blight, Bacterial Spot, and Leaf Miner infestation. Farmers usually detect these diseases by eye, which is slow and often wrong, especially for farmers who are not trained in plant pathology.

This project builds "**Plant Health**" — a mobile application built in Flutter that lets any farmer take or upload a photo of a leaf and instantly get a diagnosis and treatment suggestion, powered by Google's Gemini vision AI. Instead of training and deploying a custom detection model, the app uses Gemini's vision-language capability directly, which lets the diagnosis logic (disease identification, severity assessment, treatment advice) be developed, tested, and improved through prompt design rather than a lengthy model-training cycle — while still returning structured, localized results a farmer can act on.

2. Background Study

Plant disease is a major cause of crop loss around the world. In Bangladesh, tomato farming is common but many farmers do not have easy access to agricultural experts. A wrong or late diagnosis can lead to the wrong pesticide being used, wasted money, and lower harvest.

At the same time, smartphones are now common even in rural areas, and recent progress in cloud-based vision-language AI (like Google Gemini) makes it possible to build a tool that gives instant, understandable answers from a simple photo — no expert needed on site, and no need to train and maintain a custom computer-vision model.

3. Problem Statement

- Manual disease detection by farmers is slow, inconsistent, and error-prone.
- Expert agricultural help is not always available, especially in rural areas.
- Existing research models (CNN/YOLO based, see Literature Review) report strong accuracy but are usually tested only in a lab/notebook environment and are rarely turned into something a farmer can use on a phone.

- There is a need for a system that is both **usable** (a simple mobile app that works in real conditions, including patchy internet and possible service outages) and **understandable** (results and advice in the farmer's own language) — without requiring a custom-trained model to be built and maintained first.

4. Objectives

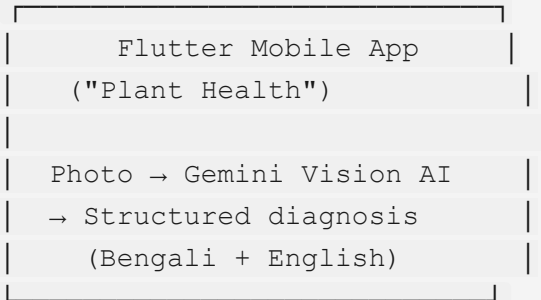
- To build a cross-platform mobile application (Android/iOS) that any farmer can use to photograph a leaf and get an instant, understandable result in Bengali.
- To use a general-purpose cloud vision AI (Google Gemini) as the diagnosis engine, avoiding the cost and lead time of collecting, annotating, and training a custom detection model.
- To design a prompting and parsing approach that reliably extracts disease status, disease name, severity, symptoms, and treatment advice from the AI's response.
- To make the mobile application reliable in real-world conditions — handling network failures, server downtime, and API key expiry gracefully instead of crashing.
- To evaluate the diagnostic accuracy of the Gemini-based approach against known leaf images and compare it against existing published work.

5. Literature Review

Author(s)	Year	Model/Approach	Achievements	Gaps/Limitations
Rajamohanam, R., & Latha, B. C.	2023	Optimized YOLOv5	93% accuracy in tomato leaf disease classification	Background interference, limited to two disease categories
Abulizi, A., Ye, J., & Guo, W.	2024	DM-YOLO (YOLOv9)	Precision increased by 2–3% vs. improved models	Small lesions and overlapping symptoms remain challenging
Tm, P., Pranathi, A., & Sai Ashritha, K.	2018	LeNet CNN	94–95% accuracy with minimal computing resources	Limited to simpler CNN architectures
Shanthi, D. L., Vinutha, K., & Ashwini, N.	2024	Custom CNN (AlexNet, VGGNet-16)	Improved classification accuracy	High computational demand for large datasets
Altalak, M., Uddin, M. A., Alajmi, A., & Rizg, A.	2022	Hybrid CNN, SVM, CBAM	97.2% accuracy in disease detection	Needs improvement in the learning process
Agarwal, M., Gupta, S. K., & Biswas, K. K.	2020	Simplified CNN (8 hidden layers)	98.4% accuracy on PlantVillage dataset	Needs more image-preprocessing optimization

Summary: Most existing work reports high accuracy but is trained/tested only on lab datasets and simple, uncrowded images, and requires the authors to collect, annotate, and train a dedicated model. Very few studies discuss turning the model into a usable, real-time farmer-facing product. This project addresses that gap differently: instead of training a new custom model, it uses an existing large-scale vision AI (Gemini) as the diagnosis engine, and puts the engineering effort into making the surrounding mobile application usable and reliable for real farmers.

6. System Overview



The application is the whole deliverable: a working, installable proof that a farmer can photograph a diseased leaf and receive an instant, localized diagnosis and treatment suggestion, using Google's Gemini vision AI as the detection engine.

7. Methodology

The mobile app follows a simple, repeatable flow:

```
Open app → Login/Signup (Firebase Auth) → Home
  → Take photo / Choose from gallery
  → Preview image → Tap "Analyze"
  → Image sent to Gemini for analysis
  → Result shown: healthy / disease name, severity, symptoms, and
  suggested medicine
```

Development approach: - Built using **Flutter**, so the same codebase runs on both Android and iOS. - Follows **Clean Architecture** (Presentation → Domain → Data layers) so the AI backend could be swapped for a different provider or a future on-device model without rewriting the UI. - Uses the **BLoC pattern** to manage app state (loading, success, error) in a predictable way. - The prompt sent to Gemini is engineered in Bengali and specifically asks for: disease status, disease name, severity, symptoms, recommended medicines with dosage, and care steps. The response is parsed into a structured result model in the data layer.

8. Technology Stack

- Flutter & Dart
- BLoC (`flutter_bloc`) for state management

- GetIt for dependency injection
- Firebase Authentication (login/signup/profile)
- Firebase Remote Config (manages AI API keys remotely, without releasing a new app version)
- Google Gemini API (`gemini-flash-latest`) for image-based disease analysis
- HTTP package for networking
- Image Picker (camera & gallery access)
- Open-Meteo API for weather information

9. System Design & Architecture

The app is organized in four layers, so each part has one clear responsibility:

```
UI (Pages/Widgets)
  ↓ user action
BLoC (Events → States)
  ↓
UseCase (business rule: "analyze this image")
  ↓
Repository (defines the contract)
  ↓
DataSource (talks to the Gemini API over HTTPS)
```

9.1 Reliable API Key Management

Because a single AI API key can expire, get rate-limited, or be revoked, the app supports a **pool of keys** instead of one: - Keys are supplied through Firebase Remote Config (so they can be rotated without publishing a new app update) or through a build-time configuration for development. - If a request fails because a key is expired, blocked, or out of quota, the app automatically retries with the next key in the pool — invisible to the user. - If the AI server itself is temporarily overloaded (a server-side error, not a key problem), the app retries the same key with a short delay, instead of wasting the whole key pool.

9.2 Error Handling

The data layer distinguishes between different failure types and shows the user a clear, local-language message instead of a technical crash: - Request timeout / no internet connection -

Malformed or unexpected server response - Image blocked by AI safety filters - All configured keys exhausted - AI server temporarily busy

This makes the app resilient in the low-connectivity conditions common in rural field use — a key requirement given the target users are farmers, not developers.

10. Implementation Details

Key screens/flows built: - **Splash screen** — initial load and Firebase setup - **Login / Signup / Profile** — Firebase Authentication - **Home navigation** — entry point to the detection feature - **Plant Detection page** — camera/gallery image capture and preview - **Analysis flow** — sends the image to the Gemini backend and shows a loading state - **Result page** — shows health status, disease name (Bengali & English), severity, symptoms, and recommended medicine/dosage

The AI prompt sent to Gemini is written in Bengali and specifically asks for: disease status, disease name, severity, symptoms, recommended medicines with dosage, and care steps — matching the target users' language.

11. Results and Evaluation

11.1 Diagnostic Accuracy Evaluation (Gemini-based)

[FILL IN — replace all bracketed values below with your real test numbers]

Metric	Value
Test images evaluated	[X]
Correct diagnoses (disease/healthy status matched ground truth)	[X] / [Total] ([X]%)
Incorrect / misclassified	[X]
Average response time per image	[X] seconds

Per-class breakdown:

Class	Images Tested	Correctly Identified	Accuracy
Healthy	[X]	[X]	[X]%
Early Blight	[X]	[X]	[X]%
Bacterial Spot	[X]	[X]	[X]%
Leaf Miner	[X]	[X]	[X]%

[Add 1–2 sentences here once the numbers are filled in, describing which class performed best/worst and why, similar to how the Literature Review discusses failure cases.]

11.2 Comparative Analysis

Study	Architecture	Disease Classes	Performance	Deployment Focus
Tm et al. (2018)	LeNet CNN	Multiple	94–95% Accuracy	Limited computing
Agarwal et al. (2020)	Simplified CNN	Multiple	98.4% Accuracy	Lab/static dataset
Rajamohanan (2023)	YOLOv5	Multiple	93% Accuracy	Real-time field
Abulizi et al. (2025)	DM-YOLO (v9)	Multiple	95.1% mAP	Real-time precision
Wang & Liu (2025)	TomatoGuard (v10)	Multiple	94.23% mAP50	High speed (129 FPS)
Shen et al. (2025)	Optimized YOLOv8	Multiple	+2.2% Precision	Feature extraction
Our Approach (2026)	Gemini Vision AI (cloud, zero-shot)	Multiple	[X]% accuracy [FILL IN]	Cloud-based, no training required

Unlike the prior work above, our approach does not train a dedicated detection model — it relies on a general-purpose cloud vision-language model. This trades control over raw detection accuracy for a large reduction in development time and no dataset/training requirement, while still needing internet connectivity and a third-party API to function.

11.3 Mobile Application Testing

The application was verified through functional testing of the full user flow (image capture → analysis → result display) and targeted testing of failure conditions: - Simulated API key

expiry/rejection → confirmed automatic rotation to the next key - Simulated server error → confirmed retry-with-backoff behavior - Simulated no-internet condition → confirmed a clear, user-facing error message instead of a crash

Automated unit/widget test coverage is limited at this stage (default Flutter widget test only); expanding automated test coverage is listed under Future Work.

12. Working Schedule (Gantt Chart)

Task	Duration	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
Problem Identification	1 week	✓									
Literature Review	1 week		✓								
Mobile App Architecture & Setup	2 weeks			✓	✓						
Gemini Integration & Prompt Design	3 weeks					✓	✓	✓			
Testing and Validation	1 week								✓		
Mobile App Development	(ongoing, parallel)										
Report/Paper Organization	2 weeks									✓	✓

13. Limitations

- The app depends entirely on a third-party cloud AI service (Gemini); it does not work offline and is subject to that provider's availability, rate limits, and pricing.
- Diagnostic accuracy is bounded by Gemini's general-purpose vision capability rather than a model fine-tuned specifically on tomato leaf disease images, so results may be less consistent than a well-trained specialist model on crowded/ambiguous symptoms.
- The evaluation test set used above is small; a larger, more diverse test set would give a more reliable accuracy estimate.
- Automated software testing (unit/integration tests) for the mobile app is minimal at this stage.

14. Expected Outcome and Future Work

- Expand the diagnostic evaluation to a larger, more diverse set of real field images to get a more reliable accuracy estimate.
- Explore fine-tuning or few-shot prompting strategies to improve Gemini's accuracy on harder classes (e.g. diseases with small, crowded lesions).
- Investigate a lightweight on-device model as an offline fallback for areas with no internet access, without removing the cloud-based option.
- Add automated test coverage for the mobile application.
- Pilot the app with real farmers to collect feedback on usability and translation accuracy.

15. Conclusion

This project delivers "Plant Health," a working, farmer-facing Flutter mobile application that provides instant, Bengali-language tomato leaf disease diagnosis and treatment advice using Google's Gemini vision AI. By using an existing cloud vision-language model instead of training a custom detector, the project focuses its engineering effort on making the application itself reliable and usable in real-world, low-connectivity conditions — while remaining architected so that the AI backend can be extended or replaced (e.g., with a fine-tuned or on-device model) in the future.

16. References

1. Agarwal, M., Gupta, S. K., & Biswas, K. K. (2020). Development of efficient CNN model for tomato crop disease identification. *Sustainable Computing: Informatics and Systems*, 28, 100407.
2. Ahmed, S., Hasan, M. B., Ahmed, T., Sony, M. R. K., & Kabir, M. H. (2022). Less is more: Lighter and faster deep neural architecture for tomato leaf disease classification. *IEEE Access*, 10, 68868–68884.
3. Altalak, M., Uddin, M. A., Alajmi, A., & Rizg, A. (2022). A hybrid approach for the detection and classification of tomato leaf diseases. *Applied Sciences*, 12(16), 8182.
4. Attallah, O. (2023). Tomato leaf disease classification via compact convolutional neural networks with transfer learning and feature selection. *Horticulturae*, 9(2), 149.
5. Baek, E. T. (2025). Attention score-based multi-vision transformer technique for plant disease classification. *Sensors*, 25(1), 270.

6. Borugadda, P., Lakshmi, R., & Sahoo, S. (2023). Transfer learning VGG16 model for classification of tomato plant leaf diseases: A novel approach for multi-level dimensional reduction. *Pertanika Journal of Science & Technology*, 31(2).
7. Rajamohanam, R., & Latha, B. C. (2023). Optimized YOLOv5 for tomato leaf disease classification.
8. Abulizi, A., Ye, J., & Guo, W. (2024). DM-YOLO: An improved YOLOv9-based detector.
9. Tm, P., Pranathi, A., & Sai Ashritha, K. (2018). LeNet-based CNN for plant disease classification.
10. Shanthi, D. L., Vinutha, K., & Ashwini, N. (2024). Custom CNN architectures (AlexNet, VGGNet-16) for tomato disease classification.